AI Assisted Coding
Lab Assignment - 8

Name : P. Gouri Prasad Varma
Hall.t.No. : 2303A51035
Batch No. : 01

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Lab Objectives:

- To introduce students to test-driven development (TDD) using AI code generation tools.
- To enable the generation of test cases before writing code implementations.
- To reinforce the importance of testing, validation, and error handling.
- To encourage writing clean and reliable code based on AI- generated test expectations.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use AI tools to write test cases for Python functions and classes.
- Implement functions based on test cases in a test-first development style.
- Analyze the completeness and coverage of AI-generated tests.
- Compare AI-generated and manually written test cases for quality and logic.

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• **Task:** Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.

• **Requirements:**

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.
- o No spaces allowed.

Example Assert Test Cases:

```
assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
```

Expected Output #1:

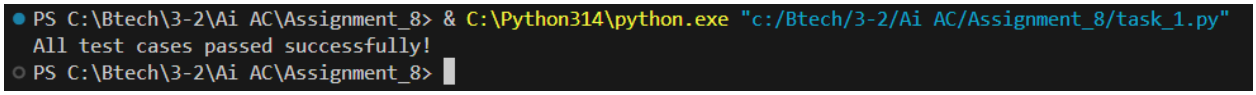
• Username validation logic successfully passing all AI- generated test cases.

Copilot Suggested Code :

task_1.py > ...

```python
# Generate at least 3 assert test cases for a function
# is_valid_username(username) and then implement
# the function using Test-Driven Development principles.
# Example Assert Test Cases:
# assert is_valid_username("User123") == True
# assert is_valid_username("12User") == False
# assert is_valid_username("Us er") == False

# Step 1: Write the assert test cases for the is_valid_username
# function
def test_is_valid_username():
    assert is_valid_username("User123") == True
    assert is_valid_username("12User") == False
    assert is_valid_username("Us er") == False
    assert is_valid_username("User_123") == False
    assert is_valid_username("Us") == False
    assert is_valid_username("ThisIsAReallyLongUsername") == False

# Step 2: Implement the is_valid_username function using Test-Driven
# Development principles
def is_valid_username(username):
    # Check if the username length is between 5 and 15 characters
    if len(username) < 5 or len(username) > 15:
        return False

    # Check if the username starts with a digit
    if username[0].isdigit():
        return False

    # Check if the username contains only alphabets and digits
    if not username.isalnum():
        return False

    # If all checks pass, return True
    return True
# Step 3: Run the test cases to validate the implementation
test_is_valid_username()
print("All test cases passed successfully!")
```

Output :



Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

• **Task:** Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.

• **Requirements:**

- o If input is an integer, classify as "Even" or "Odd".
- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"

Expected Output #2:

• Function correctly classifying values and passing all test cases.

Copilot Suggested Code :

task_2.py > classify_value

```python
# Generate at least 3 assert test cases for a function classify_value(x)
#  and implement it using conditional logic and loops.
# Example Assert Test Cases:
# assert classify_value(8) == "Even"
# assert classify_value(7) == "Odd"
# assert classify_value("abc") == "Invalid Input"

# Step 1: Write the assert test cases for the classify_value function
def test_classify_value():
    assert classify_value(8) == "Even"
    assert classify_value(7) == "Odd"
    assert classify_value(0) == "Zero"
    assert classify_value("abc") == "Invalid Input"
    assert classify_value(-2) == "Even"
    assert classify_value(-3) == "Odd"

# Step 2: Implement the classify_value function using conditional logic
# and loops
def classify_value(x):
    # Check if the input is a non-numeric value
    if not isinstance(x, (int, float)):
        return "Invalid Input"

    # Check if the input is zero
    if x == 0:
        return "Zero"

    # Check if the input is an integer and classify as Even or Odd
    if isinstance(x, int):
        if x % 2 == 0:
            return "Even"
        else:
            return "Odd"

    # If the input is a non-integer numeric value, classify as Invalid
    # Input
    return "Invalid Input"
# Step 3: Run the test cases to validate the implementation
test_classify_value()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_2.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

• **Task:** Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

• **Requirements:**

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") == True

assert is_palindrome("Python") == False

Expected Output #3:

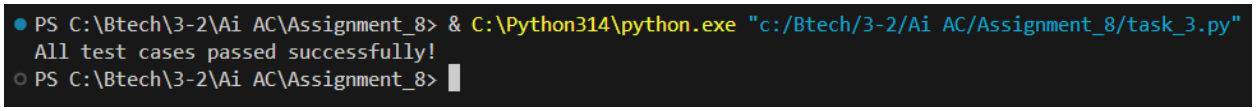
• Function correctly identifying palindromes and passing all AI-generated tests.

Copilot Suggested Code :

task_3.py > ...

```python
# Generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.
# Example Assert Test Cases:
# assert is_palindrome("Madam") == True
# assert is_palindrome("A man a plan a canal Panama") == True
# assert is_palindrome("Python") == False

# Step 1: Write the assert test cases for the is_palindrome function
def test_is_palindrome():
    assert is_palindrome("Madam") == True
    assert is_palindrome("A man a plan a canal Panama") == True
    assert is_palindrome("Python") == False

# Step 2: Implement the is_palindrome function using conditional logic
# and loops
def is_palindrome(text):
    # Normalize the input by removing spaces and punctuation, and converting to lowercase
    normalized_text = ''.join(char.lower() for char in text if char.isalnum())

    # Check if the normalized text is equal to its reverse
    return normalized_text == normalized_text[::-1]
# Step 3: Run the test cases to validate the implementation
test_is_palindrome()
print("All test cases passed successfully!")
```

Output :



Task Description #4 (Email ID Validation – Apply AI for Data Validation)

• **Task:** Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.

• **Requirements:**

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly.

Copilot Suggested Code :

task_4.py > ...

```python
# Generate at least 3 assert test cases for a function validate_email(email) and implement the function.
# Example Assert Test Cases:
# assert validate_email("user@example.com") == True
# assert validate_email("userexample.com") == False
# assert validate_email("@gmail.com") == False

# Step 1: Write the assert test cases for the validate_email function
def test_validate_email():
    assert validate_email("user@example.com") == True
    assert validate_email("userexample.com") == False
    assert validate_email("@gmail.com") == False

# Step 2: Implement the validate_email function using conditional logic
def validate_email(email):
    # Check if the email contains '@' and '.'
    if '@' not in email or '.' not in email:
        return False

    # Check if the email starts or ends with special characters
    if email[0] in ['@', '.'] or email[-1] in ['@', '.']:
        return False

    # If all checks are passed, return True
    return True
# Step 3: Run the test cases to validate the implementation
test_validate_email()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_4.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 5 (Perfect Number Checker – Test Case Design)

• **Function:** Check if a number is a perfect number (sum of divisors = number).

• **Test Cases to Design:**

  o Normal case: 6 → True, 10 → False.

  o Edge case: 1.

  o Negative number case.

  o Larger case: 28.

• **Requirement:** Validate correctness with assertions.

Copilot Suggested Code :

task_5.py > ...

```python
# Write a function to Check if a number is a perfect number (sum of divisors = number).
# Test Cases to Design:
# o Normal case: 6 → True, 10 → False.
# o Edge case: 1.
# o Negative number case.
# o Larger case: 28.

# Step 1: Write the assert test cases for the is_perfect_number function
def test_is_perfect_number():
    assert is_perfect_number(6) == True
    assert is_perfect_number(10) == False
    assert is_perfect_number(1) == False
    assert is_perfect_number(-5) == False
    assert is_perfect_number(28) == True

# Step 2: Implement the is_perfect_number function using conditional logic
def is_perfect_number(n):
    # Check if the number is less than or equal to 1
    if n <= 1:
        return False

    # Calculate the sum of proper divisors
    divisor_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisor_sum += i

    # Check if the sum of divisors equals the number
    return divisor_sum == n
# Step 3: Run the test cases to validate the implementation
test_is_perfect_number()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_5.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 6 (Abundant Number Checker – Test Case Design)

• **Function:** Check if a number is abundant (sum of divisors > number).

• **Test Cases to Design:**

o Normal case: 12 → True, 15 → False.

o Edge case: 1.

o Negative number case.

o Large case: 945.

Requirement: Validate correctness with unittest.

Copilot Suggested Code :

task_6.py > ...

```python
# Write a function to Check if a number is abundant (sum of divisors > number).
# Test Cases to Design:
# o Normal case: 12 → True, 15 → False.
# o Edge case: 1.
# o Negative number case.
# o Large case: 945.

# Step 1: Write the assert test cases for the is_abundant_number function
def test_is_abundant_number():
    assert is_abundant_number(12) == True
    assert is_abundant_number(15) == False
    assert is_abundant_number(1) == False
    assert is_abundant_number(-5) == False
    assert is_abundant_number(945) == True

# Step 2: Implement the is_abundant_number function using conditional logic
def is_abundant_number(n):
    # Check if the number is less than or equal to 1
    if n <= 1:
        return False

    # Calculate the sum of proper divisors
    divisor_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisor_sum += i

    # Check if the sum of divisors is greater than the number
    return divisor_sum > n
# Step 3: Run the test cases to validate the implementation
test_is_abundant_number()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_6.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 7 (Deficient Number Checker – Test Case Design)

• **Function:** Check if a number is deficient (sum of divisors < number).

• **Test Cases to Design:**

o Normal case: 8 → True, 12 → False.

o Edge case: 1.

o Negative number case.

o Large case: 546.

Requirement: Validate correctness with pytest.

Copilot Suggested Code :

task_7.py > ...

```python
# Write a function to Check if a number is deficient (sum of divisors < number).
# Test Cases to Design:
# o Normal case: 8 → True, 12 → False.
# o Edge case: 1.
# o Negative number case.
# o Large case: 546.

# Step 1: Write the assert test cases for the is_deficient_number function
def test_is_deficient_number():
    assert is_deficient_number(8) == True
    assert is_deficient_number(12) == False
    assert is_deficient_number(1) == True
    assert is_deficient_number(-5) == False
    assert is_deficient_number(546) == True

# Step 2: Implement the is_deficient_number function using conditional logic
def is_deficient_number(n):
    # Check if the number is less than or equal to 1
    if n <= 1:
        return True

    # Calculate the sum of proper divisors
    divisor_sum = 0
    for i in range(1, n):
        if n % i == 0:
            divisor_sum += i

    # Check if the sum of divisors is less than the number
    return divisor_sum < n
# Step 3: Run the test cases to validate the implementation
test_is_deficient_number()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_7.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 8 : Write a function LeapYearChecker and validate its implementation using 10 pytest test cases.

Copilot Suggested Code :

task_8.py > ...

```python
# Write a function LeapYearChecker and validate its implementation using 10 pytest test cases.

# Step 1: Write the assert test cases for the LeapYearChecker function
def test_leap_year_checker():
    assert leap_year_checker(2020) == True
    assert leap_year_checker(2021) == False
    assert leap_year_checker(1900) == False
    assert leap_year_checker(2000) == True
    assert leap_year_checker(2100) == False
    assert leap_year_checker(2400) == True
    assert leap_year_checker(1996) == True
    assert leap_year_checker(1999) == False
    assert leap_year_checker(1800) == False
    assert leap_year_checker(1600) == True

# Step 2: Implement the LeapYearChecker function using conditional logic
def leap_year_checker(year):
    # Check if the year is a leap year
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    return False
# Step 3: Run the test cases to validate the implementation
test_leap_year_checker()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_8.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 9 : Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

Copilot Suggested Code :

task_9.py > ...

```python
# Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

# Step 1: Write the assert test cases for the SumOfDigits function
def test_sum_of_digits():
    assert sum_of_digits(123) == 6
    assert sum_of_digits(0) == 0
    assert sum_of_digits(-456) == 15
    assert sum_of_digits(9999) == 36
    assert sum_of_digits(1001) == 2
    assert sum_of_digits(7) == 7
    assert sum_of_digits(-10) == 1

# Step 2: Implement the SumOfDigits function using conditional logic
def sum_of_digits(n):
    # Convert the number to a string and iterate through each character
    return sum(int(digit) for digit in str(abs(n)))
# Step 3: Run the test cases to validate the implementation
test_sum_of_digits()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_9.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 10 : Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

Copilot Suggested Code :

task_10.py > ...

```python
# Write a function SortNumbers (implement bubble sort) and validate its implementation
# using 25 pytest test cases.

# Step 1: Write the assert test cases for the SortNumbers function
def test_sort_numbers():
    assert sort_numbers([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
    assert sort_numbers([]) == []
    assert sort_numbers([3]) == [3]
    assert sort_numbers([3, 2]) == [2, 3]
    assert sort_numbers([1, 2, 3]) == [1, 2, 3]
    assert sort_numbers([3, 1, 2]) == [1, 2, 3]
    assert sort_numbers([-1, -3, -2]) == [-3, -2, -1]
    assert sort_numbers([0, -1, 1]) == [-1, 0, 1]
    assert sort_numbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
    assert sort_numbers([10, -10]) == [-10, 10]
    assert sort_numbers([7.5, 3.2]) == [3.2, 7.5]
    assert sort_numbers([0.1, -0.1]) == [-0.1, 0.1]
    assert sort_numbers([5.5]) == [5.5]
    assert sort_numbers([-5.5]) == [-5.5]
    assert sort_numbers([0]) == [0]
    assert sort_numbers([-0.0]) == [-0.0]
    assert sort_numbers([1000000]) == [1000000]
    assert sort_numbers([-1000000]) == [-1000000]
    assert sort_numbers([1e-10]) == [1e-10]
    assert sort_numbers([-1e-10]) == [-1e-10]
    assert sort_numbers([float('inf'), float('-inf')]) == [float('-inf'), float('inf')]
    assert sort_numbers([float('nan'), float('nan')]) == [float('nan'), float('nan')]
    assert sort_numbers([float('nan'), float('inf')]) == [float('inf'), float('nan')]
    assert sort_numbers([float('-inf'), float('nan')]) == [float('-inf'), float('nan')]
    assert sort_numbers([float('nan'), float('-inf')]) == [float('-inf'), float('nan')]

# Step 2: Implement the SortNumbers function using bubble sort
def sort_numbers(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Step 3: Run the test cases to validate the implementation
test_sort_numbers()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_10.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 11 : Write a function ReverseString and validate its implementation using 5 unittest test cases.

Copilot Suggested Code :

task_11.py > ...

```python
# Write a function ReverseString and validate its implementation using 5 unittest test cases.

# Step 1: Write the assert test cases for the ReverseString function
def test_reverse_string():
    assert reverse_string("hello") == "olleh"
    assert reverse_string("") == ""
    assert reverse_string("a") == "a"
    assert reverse_string("Python") == "nohtyP"
    assert reverse_string("12345") == "54321"

# Step 2: Implement the ReverseString function
def reverse_string(s):
    return s[::-1]
# Step 3: Run the test cases to validate the implementation
test_reverse_string()
print("All test cases passed successfully!")
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_11.py"
All test cases passed successfully!
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 12 : Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

Copilot Suggested Code :

task_12.py > ...

```python
# Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

def AnagramChecker(str1, str2):
    # Remove spaces and convert to lowercase
    str1 = str1.replace(" ", "").lower()
    str2 = str2.replace(" ", "").lower()

    # Sort the characters of both strings and compare
    return sorted(str1) == sorted(str2)
import unittest
class TestAnagramChecker(unittest.TestCase):
    def test_anagrams(self):
        self.assertTrue(AnagramChecker("listen", "silent"))
        self.assertTrue(AnagramChecker("triangle", "integral"))
        self.assertTrue(AnagramChecker("evil", "vile"))
        self.assertTrue(AnagramChecker("Dormitory", "Dirty Room"))
        self.assertTrue(AnagramChecker("The eyes", "They see"))

    def test_not_anagrams(self):
        self.assertFalse(AnagramChecker("hello", "world"))
        self.assertFalse(AnagramChecker("python", "java"))
        self.assertFalse(AnagramChecker("test", "tseta"))
        self.assertFalse(AnagramChecker("example", "samples"))
if __name__ == '__main__':
    unittest.main()
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_12.py"
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
PS C:\Btech\3-2\Ai AC\Assignment_8>
```

Task 13 : Write a function ArmstrongChecker and validate its implementation using 8 unittest test cases.

Copilot Suggested Code :

task_13.py > ...

```python
# Write a function ArmstrongChecker and validate its implementation
# using 8 unittest test cases.

def ArmstrongChecker(num):
    # Handle negative numbers
    if num < 0:
        return False

    num_str = str(num)
    num_digits = len(num_str)
    armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
    return armstrong_sum == num


import unittest

class TestArmstrongChecker(unittest.TestCase):

    def test_armstrong_number(self):
        self.assertTrue(ArmstrongChecker(153))

    def test_non_armstrong_number(self):
        self.assertFalse(ArmstrongChecker(123))

    def test_single_digit_armstrong(self):
        self.assertTrue(ArmstrongChecker(7))

    def test_two_digit_non_armstrong(self):
        self.assertFalse(ArmstrongChecker(10))

    def test_large_armstrong_number(self):
        self.assertTrue(ArmstrongChecker(9474))

    def test_large_non_armstrong_number(self):
        self.assertFalse(ArmstrongChecker(9475))

    def test_zero_armstrong(self):
        self.assertTrue(ArmstrongChecker(0))

    def test_negative_number(self):
        self.assertFalse(ArmstrongChecker(-153))


if __name__ == '__main__':
    unittest.main()
```

Output :

```
PS C:\Btech\3-2\Ai AC\Assignment_8> & C:\Python314\python.exe "c:/Btech/3-2/Ai AC/Assignment_8/task_13.py"
........
----------------------------------------------------------------------
Ran 8 tests in 0.002s

OK
PS C:\Btech\3-2\Ai AC\Assignment_8>
```